

CertiGuard SDK

On-Premise Software License Protection
Defense-in-Depth: A 10-Layer Architecture

Prevention · Detection · Forensics · Offline-First

InnovIA ESI Hackathon — April 2026

L10:	Audit Log & Evidence Chain
L9:	Watermarking & Attribution
L8:	Behavioral AI Detection
L7:	Honeypot Tripwires
L6:	PoW Dead Man's Switch
L5:	Anti-Debug & Anti-Tamper
L4:	Challenge-Response Verifier
L3:	Installation DNA & Counter
L2:	Hardware Fingerprint
L1:	Ed25519 Signature Core

CertiGuard 10-layer vertical defense stack

Contents

1	System Overview	2
1.1	The Problem	2
1.2	Why Existing Solutions Fail	2
1.3	CertiGuard's Approach	2
2	Architecture Layers	2
2.1	Layer Stack Summary	3
2.2	Complete Layer Catalogue	3
3	Data Flow Diagram	3
4	Ed25519 Cryptographic Core	4
4.1	Mathematical Foundation	4
4.2	Why This Beats Denuvo-Style Decrypt-at-Runtime Patterns	4
4.3	License File Structure	4
5	Installation DNA — Anti-Cloning	5
5.1	The VM Cloning Problem	5
5.2	Three DNA Components	5
5.3	Why All Three Cannot Be Cloned Simultaneously	5
6	AI Behavioral Detection	5
6.1	Isolation Forest — How It Works	5
6.2	Feature Table	6
6.3	Offline-First Design	6
7	Complete Attack-Defense Matrix	7
8	SDK Integration	7
8.1	Minimal Integration Snippet	7
8.2	Component Layout	8
8.3	Deployment Scenarios	8
9	References	8

1 System Overview

1.1 The Problem

On-premise software is frequently attacked through license-file editing, debugger-assisted binary patching, virtual-machine cloning, and usage abuse. Traditional single-check licensing fails because attackers need to bypass only one gate. CertiGuard addresses this with a layered architecture where cryptographic trust, host binding, runtime verification, behavioral detection, and forensic evidence are chained into one offline-first control plane.

1.2 Why Existing Solutions Fail

Table 1: Limitations of common licensing approaches

Existing approach	Why it fails in practice
Plain serial key check	Static key checks are patched by flipping a branch instruction in minutes.
Online-only activation	Air-gapped or restricted enterprise networks cannot depend on real-time cloud checks.
Simple hardware lock	Single-factor hardware binding is bypassed by VM snapshots or partial spoofing.
Code obfuscation only	Obfuscation raises effort but does not provide cryptographic authenticity or evidence trail.
Periodic polling license server	“Kill server, keep app running” attack remains possible without local heartbeat enforcement.

1.3 CertiGuard’s Approach

CertiGuard implements a defense-in-depth model: signed license authenticity (L1), machine/installation context continuity (L2–L3), verifier challenge integrity (L4), anti-analysis and liveness controls (L5–L6), tamper tripwires (L7), local anomaly scoring (L8), leak attribution (L9), and immutable audit chaining (L10). The system remains operational offline while preserving post-event forensic visibility for vendor review.

Attack Vector	Shield Layer
File editing	L1 Ed25519
License copy / VM clone	L2 Hardware + L3 DNA
Debug attach	L5 Anti-Debug
Snapshot rollback	L3 Counter
Over-usage	L8 AI Detection

2 Architecture Layers

2.1 Layer Stack Summary

#	Name	Phase	Key Role
1	Ed25519 Signing	Prevention	Authenticity anchor
2	Hardware Fingerprint	Prevention	Host binding
3	Installation DNA	Prevention	Anti-clone / anti-rollback
4	Separate Verifier	Prevention	Binary patch resistance
5	Anti-Debug	Detection	Reverse-eng blocker
6	Dead Man's Switch	Detection	Liveness heartbeat
7	Honeypot Fields	Detection	Edit tripwire
8	Behavioral AI	Detection	Anomaly scoring
9	Watermarking	Forensics	Leak attribution
10	Audit Hash Chain	Forensics	Evidence ledger

2.2 Complete Layer Catalogue

Table 2: Layer model, threat focus, and offline suitability

Layer	Name	Threat Stopped	Algorithm/Method	Offline?
L1	Ed25519 Signing	File tampering, forgery	EdDSA on Curve25519	YES
L2	Hardware Fingerprint	License copying	SHA-256 + CPU/Board	YES
L3	Installation DNA	VM cloning, snapshots	UUID + Timestamp + Counter	YES
L4	Separate Verifier	Binary patching	Challenge-response HMAC	YES
L5	Anti-Debug	Debugger analysis	IsDebuggerPresent + timing	YES
L6	Dead Man's Switch	Process killing	PoW heartbeat chain	YES
L7	Honeypot Fields	License editing	Tripwire fields in payload	YES
L8	Behavioral AI	Over-usage, sharing	Isolation Forest (sklearn)	YES
L9	Watermarking	Leak tracing	Per-customer encoding	YES
L10	Audit Log	Evidence destruction	SHA-256 hash chain	YES

Design Principle

No single layer is trusted as a sole control. Runtime acceptance requires a sequence of independent controls whose outputs are both *decision signals* and *forensic events*.

3 Data Flow Diagram

The following describes the end-to-end vendor-to-client flow:

VENDOR SIDE		CLIENT SIDE
Private Key		.certiguard file
↓		↓
License Creator	USB/Email	→ Layer 1: Verify Signature
.certiguard file		↓ Layer 2: Check Hardware
		↓ Layer 3: Check DNA
		↓ Layer 4: Challenge-Response
		↓ Layer 5: Anti-Debug
		✓ Software Runs

Background threads run continuously during software execution:

- **Layer 6 (DMS):** PoW heartbeat liveness monitoring
- **Layer 8 (AI):** Behavioral anomaly scoring
- **Layer 10 (Audit):** Cryptographic event chain accumulation

4 Ed25519 Cryptographic Core

4.1 Mathematical Foundation

CertiGuard's trust anchor is vendor-side Ed25519 signing. The client validates signatures using only the public key.

Private key: $d = \text{random_bytes}(32)$

Public key: $Q = d \cdot G$

Signature: $\sigma = (R, S), \quad S = (r + k \cdot d) \bmod L$

Verification: $S \cdot G = R + k \cdot Q$

Security Implication

Without the vendor private key d , a valid signature over a modified license payload is computationally infeasible under standard Ed25519 assumptions.

4.2 Why This Beats Denuvo-Style Decrypt-at-Runtime Patterns

Table 3: Comparison: code decryption DRM vs signed-parameter architecture

Property	Denuvo DRM	CertiGuard
What's protected	Game executable code	License parameters and policy payload
Secret in RAM?	YES — decryption key material transiently present	NO — private key remains vendor-side only
Dump attack?	Possible via runtime memory dump	Not applicable; no private signing key in client runtime
Status	Historically cracked in multiple titles	Signature forgery remains mathematically infeasible

4.3 License File Structure

Listing 1: Conceptual signed license payload (all fields covered by signature)

```

1 {
2   "version": "3.0",
3   "license_id": "LIC-20260425025054",
4   "issued_to": "Customer A",
5   "issued_at": "2026-04-25T02:50:54Z",
6   "valid_until": "2027-04-25T02:50:54Z",
7   "hardware_fingerprint": "sha256(...)",
8   "install_dna": {

```

```
9   "install_id": "uuid-v4",
10  "first_boot": "2026-04-01T10:00:00Z",
11  "boot_count_at_issue": 37
12 },
13 "parameters": {
14   "max_users": 50,
15   "modules": ["core", "analytics"]
16 },
17 "tpm": { "available": true, "anchor": "optional-tpm-anchor" },
18 "watermark": "customer-bound-short-hash",
19 "tripwires": {
20   "premium_unlock": false,
21   "admin_override": false
22 }
23 }
```

5 Installation DNA — Anti-Cloning

5.1 The VM Cloning Problem

A cloned VM can preserve application files, including the license file, but cannot preserve all runtime continuity signals simultaneously. CertiGuard combines machine-bound encryption context, first-boot chronology, and monotonic counters to make state replay and snapshot rollback observable.

5.2 Three DNA Components

Table 4: DNA continuity checks under VM clone/snapshot conditions

Component	Original VM	Clone after snapshot	Result
UUID	a3f9...	Cannot decrypt (wrong HW)	REJECTED
First-Boot	2026-04-01	2026-04-15 (new boot)	MISMATCH
Boot Counter	150	100 (went backward)	REJECTED

5.3 Why All Three Cannot Be Cloned Simultaneously

A successful clone attack would require simultaneously reproducing (i) hardware-derived cryptographic context, (ii) installation timeline consistency, and (iii) monotonic counter progression. In practical enterprise environments, attackers can imitate one or two signals, but not all three without triggering at least one mismatch event and corresponding forensic evidence.

6 AI Behavioral Detection

6.1 Isolation Forest — How It Works

Isolation Forest isolates anomalous points using random partition trees; observations requiring fewer splits are more likely to be outliers. CertiGuard applies this locally on runtime usage features and persists baseline statistics to support offline enforcement.

Score Range	Normal Sessions	Fraudulent Sessions
0.04 to 0.00	High	—
−0.04 to −0.20	Moderate	—
−0.24 to −0.30	Low	—
Threshold: −0.30	— enforcement boundary —	
−0.32 to −0.44	—	High
−0.48 to −0.56	—	Moderate

6.2 Feature Table

Table 5: Behavioral features used for local anomaly scoring

Name	Normal Range	Fraud Indicator
Active users	5–60	Sudden spikes beyond licensed capacity
Hour-of-day pattern	Business-hour dominant	High activity at abnormal hours
Unique machines (24h)	Stable tenant profile	Sharp fan-out across many hosts
API calls/minute	Predictable envelope	Burst amplification and scripted cadence
CPU percent	10–75%	Sustained high load from automation loops
Memory percent	20–80%	Unusual sustained pressure with license abuse
Process count	Environment baseline	Abrupt tooling/process injection patterns
GPU/discrete hint	Stable hardware signature	Unexpected hardware class shifts

6.3 Offline-First Design

Enforcement path: local detection → audit hash chain → vendor review at renewal/sync. No continuous cloud dependency is required; evidence accumulates locally and can be synchronized opportunistically.

7 Complete Attack-Defense Matrix

Table 6: Attack-to-layer mapping across 14 scenarios

ID	Attack Description	Layer	Detection Method	Result
A1	Edit signed field in license payload	L1	Ed25519 signature verification fails	Rejected
A2	Replace <code>valid_until</code> with future date	L1	Signature mismatch on modified payload	Rejected
A3	Copy license to a different machine	L2	Hardware fingerprint mismatch	Rejected
A4	Clone VM image to another host	L2/L3	HW mismatch + DNA continuity break	Rejected
A5	Snapshot rollback to older state	L3	Monotonic counter regression	Rejected
A6	Patch executable branch around verify	L4	Challenge-response integrity fails	Rejected
A7	Swap verifier response blob	L4	HMAC challenge mismatch	Rejected
A8	Attach debugger to runtime process	L5	Anti-debug detectors + timing heuristics	Blocked
A9	Single-step runtime to extract logic	L5	Timing anomaly and debugger flags	Blocked
A10	Kill watchdog/liveness process repeatedly	L6	PoW heartbeat stale/chain invalid	Restricted
A11	Add <code>PREMIUM_UNLOCK=true</code> in payload	L7	Honeypot tripwire before full verify	Rejected
A12	Share one license across many teams	L8	Isolation Forest anomaly score crossing threshold	Flagged
A13	Leak customer license on forums	L9	Per-customer watermark attribution	Traceable
A14	Delete/modify audit evidence rows	L10	Hash-chain verification failure	Alert

8 SDK Integration

8.1 Minimal Integration Snippet

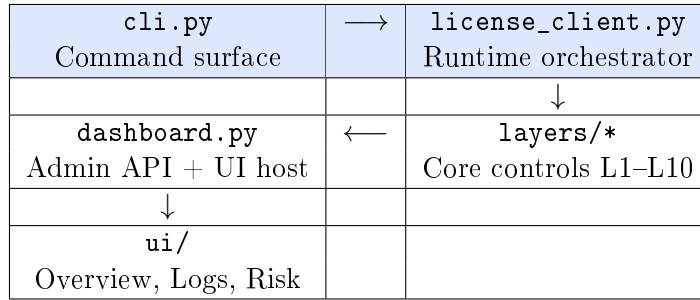
Listing 2: Three-line host application integration

```

1 from certiguard.license_client import CertiGuardClient
2 result = CertiGuardClient(state_dir).verify_runtime(
3     license_path=lic, public_key_path=pub,
4     heartbeat_key=key, behavior_features=features)
5 assert result.ok, result.code

```

8.2 Component Layout



8.3 Deployment Scenarios

Table 7: Deployment modes

Scenario	Active Layers	Internet Required
Air-gapped	L1–L10 (no external online ping dependency)	NO
Standard enterprise	All layers + optional sync at renewal	NO
Trial copy	L1 + L7 (lightweight policy mode)	NO

9 References

- [1] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B.-Y. Yang, “High-speed high-security signatures,” *Journal of Cryptographic Engineering*, 2011.
- [2] IETF, “RFC 8032: Edwards-Curve Digital Signature Algorithm (EdDSA),” 2017.
- [3] J. Brendel, C. Cremers, and D. Jackson, “A formal treatment of Ed25519 signature security,” in *IEEE Symposium on Security and Privacy*, 2021.
- [4] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” in *IEEE International Conference on Data Mining (ICDM)*, 2008.
- [5] B. De Sutter, A. Slowinska, and S. Schrittwieser, “Software protection methodologies: State of the art and open challenges,” *ACM Computing Surveys*, 2024.
- [6] S. Chow, P. A. Eisen, H. Johnson, and P. C. van Oorschot, “White-box cryptography and implementation attacks,” in *SAC*, 2002.
- [7] National Security Agency, “Trusted Platform Module (TPM) Use Cases and Deployment Guidance,” Technical Report, 2024.
- [8] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008.

Final Note

This architecture is intentionally offline-first: it prioritizes local enforceability, cryptographic authenticity, and forensic continuity while still enabling periodic vendor-side review through synchronized or transferred audit evidence.